

Day40 安全开发-JavaEE应用&SpringBoot框架 &JWT身份鉴权&打包部署JAR&WAR

① PHP

原生类



框架类



② JS

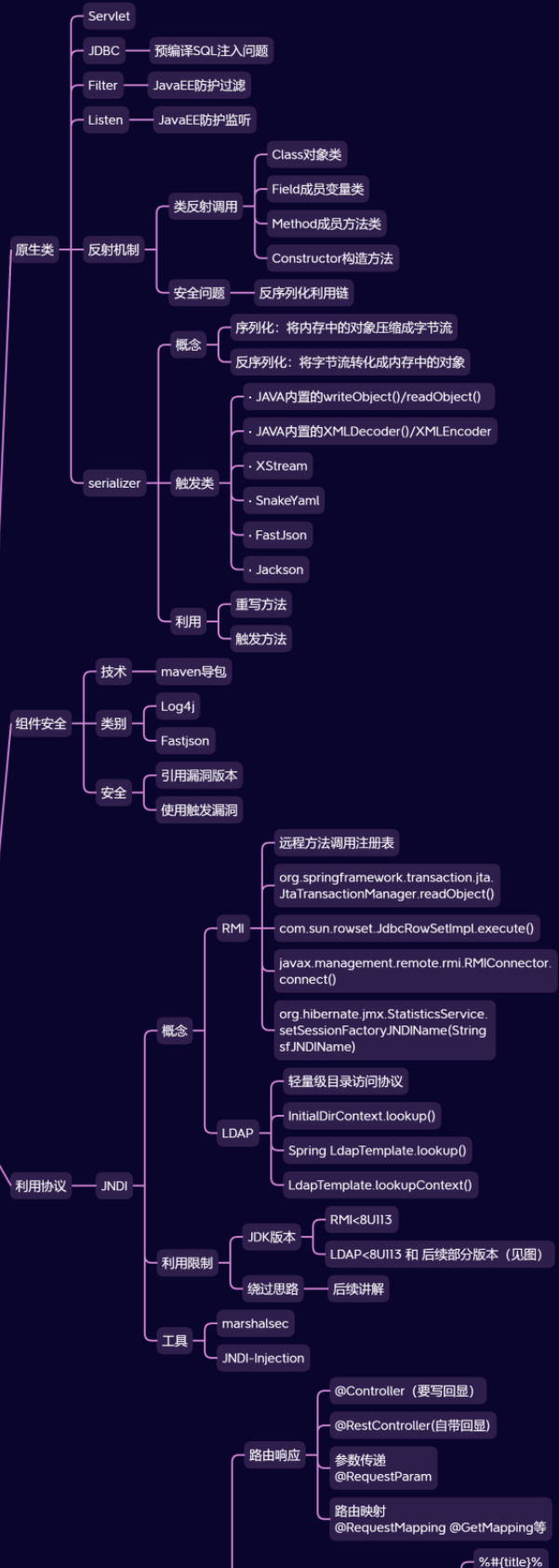
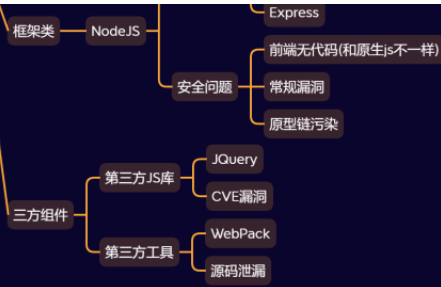
原生类

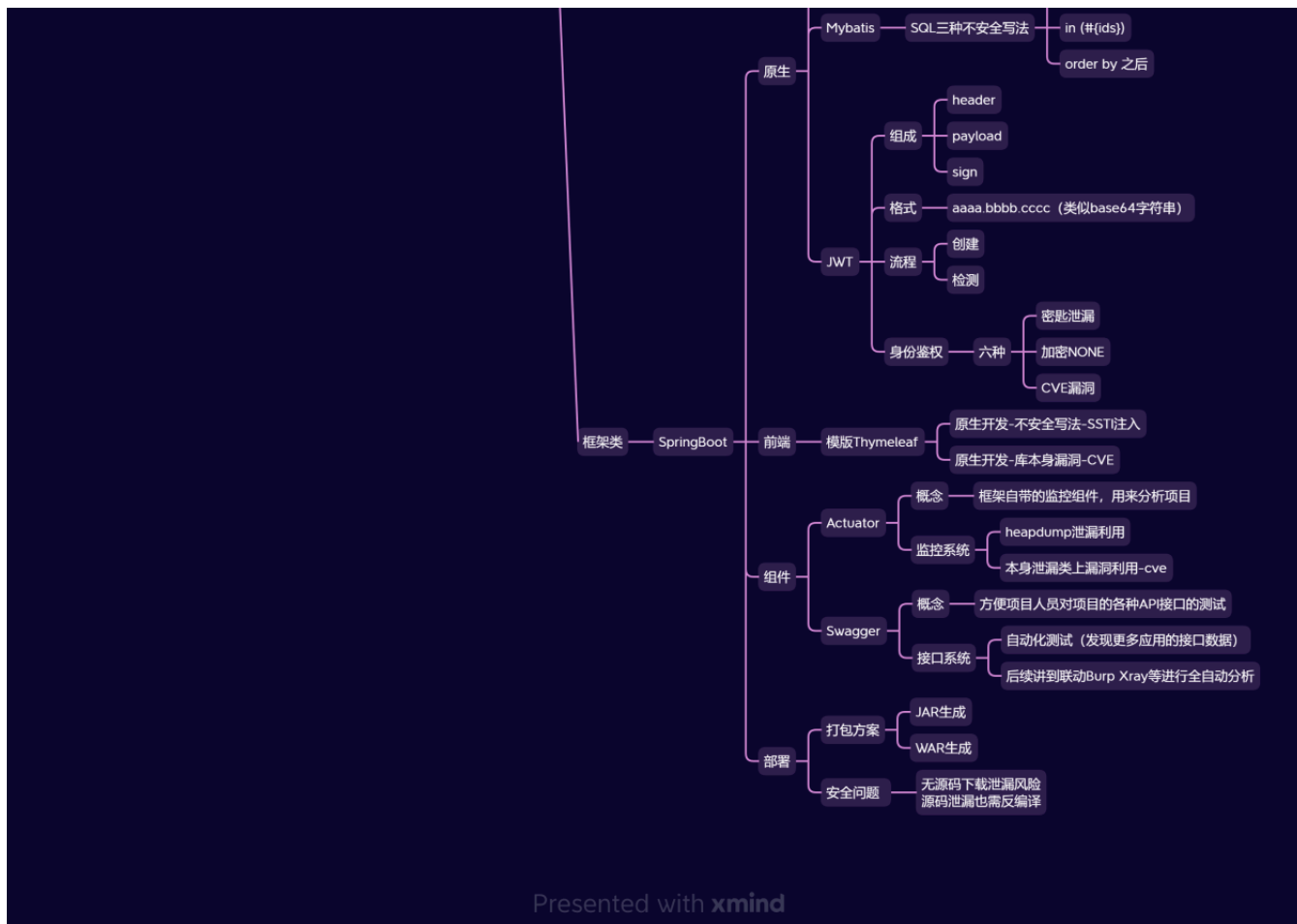




安全开发-小迪安全

③ Java





1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断

-
- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
- 2、PHP文件管理-编辑&包含功能实现

-
- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞

-
- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞

-
- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买

-
- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试

-
- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞

-
- 1、三方库-JQuery-使用&安全
 - 2、打包器-WebPack-使用&安全

-
- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis

-
- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器

-
- 1、JavaEE-反射机制-开发和安全应用场景
 - 2、JavaEE-反射机制-类&成员变量&方法&构造方法操作
 - 3、JavaEE-反射机制-安全应用&反射执行&反序列化链相关
-

- 1、JavaEE-反序列化-解释&使用&安全
- 2、JavaEE-安全-利用链&直接重写方法
- 3、JavaEE-安全-利用链&外部重写方法

- 1、JavaEE-组件安全-Log4j
- 2、JavaEE-组件安全-Fastjson
- 3、JavaEE-基本了解-JNDI-API

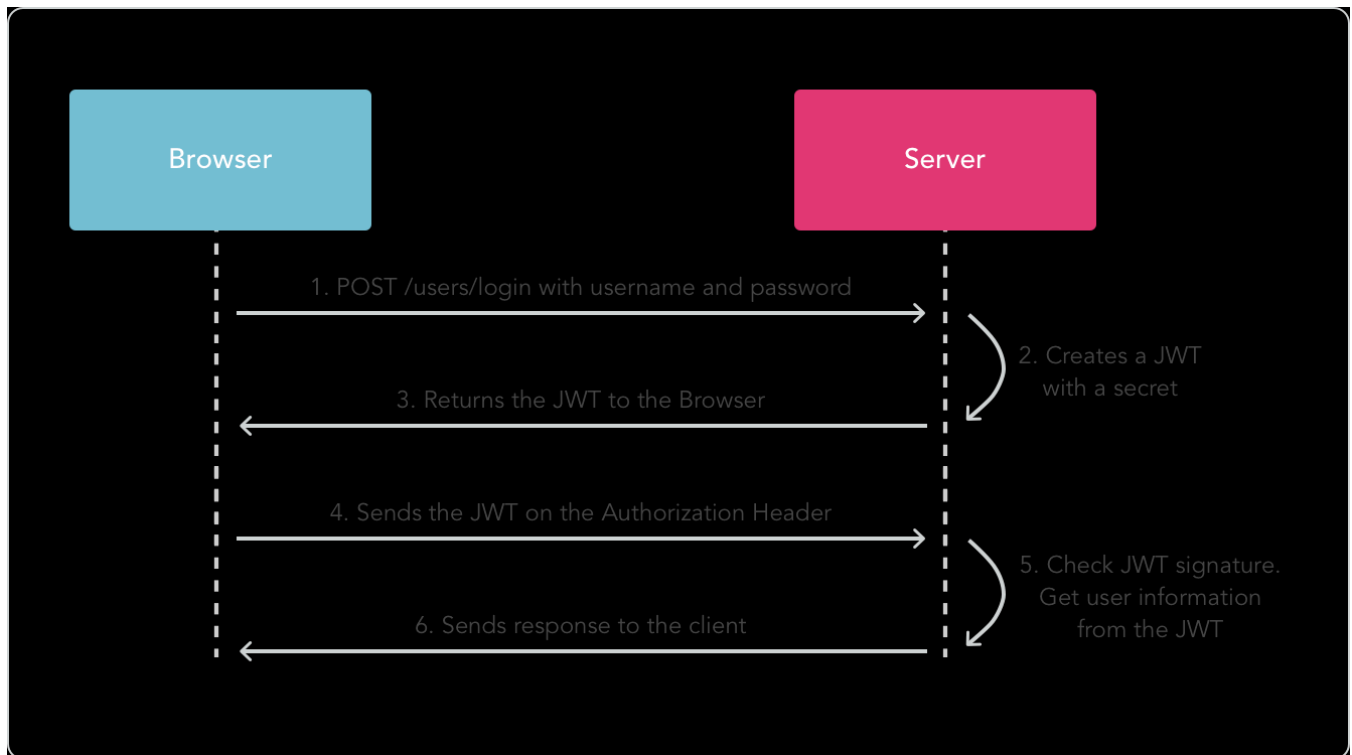
- 1、JavaEE-JNDI注入-RMI&LDAP
- 2、JavaEE-漏洞结合-FastJson链
- 3、JavaEE-漏洞条件-JDK版本绕过

- 1、JavaEE-SpringBoot-WebAPP&路由
- 2、JavaEE-SpringBoot-Mybatis&注入
- 3、JavaEE-SpringBoot-Thymeleaf&SSTI

- 1、JavaEE-SpringBoot-监控系统-Actuator
- 2、JavaEE-SpringBoot-接口系统-Swagger
- 3、JavaEE-SpringBoot-监控&接口&安全问题

- 1、JavaEE-SpringBoot-身份鉴权-JWT技术
- 2、JavaEE-SpringBoot-打包部署-JAR&WAR

2.演示案例



2.1 SpringBoot-身份鉴权-JWT技术

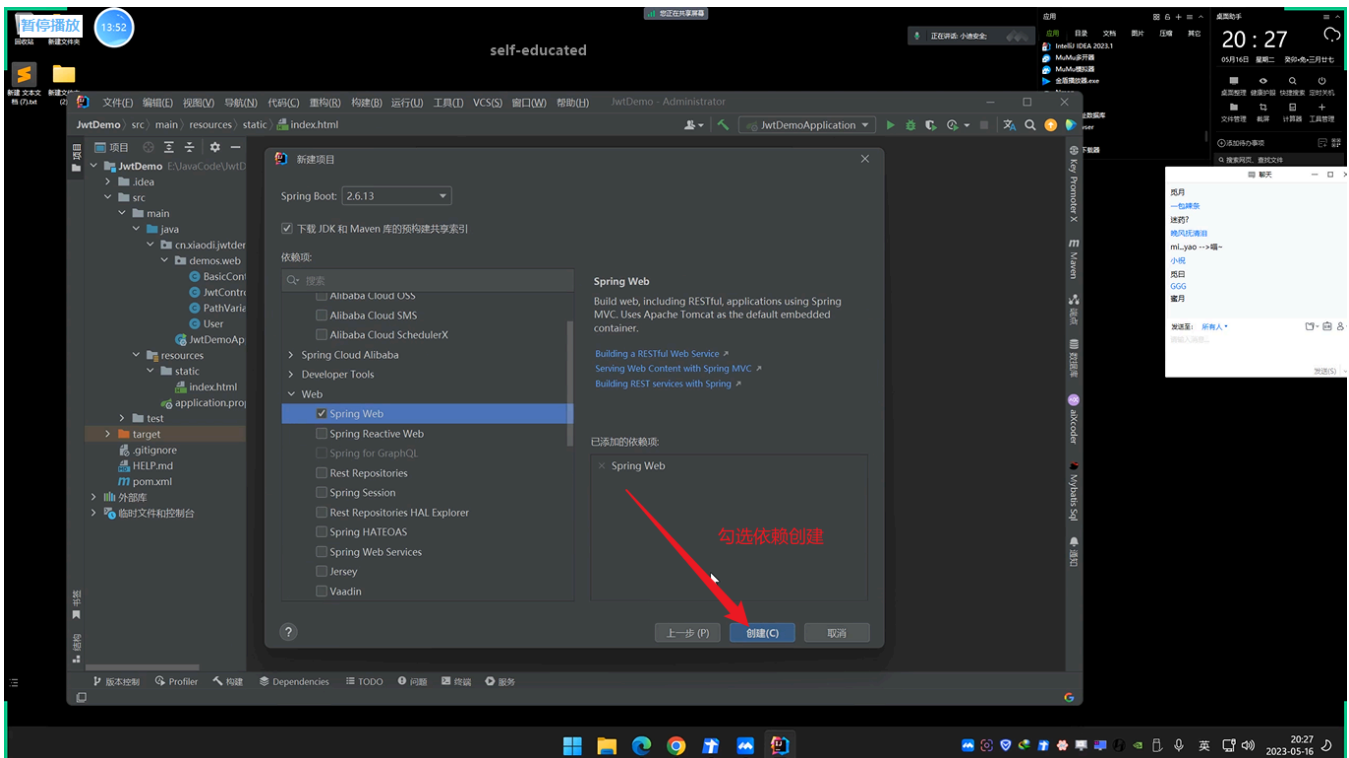
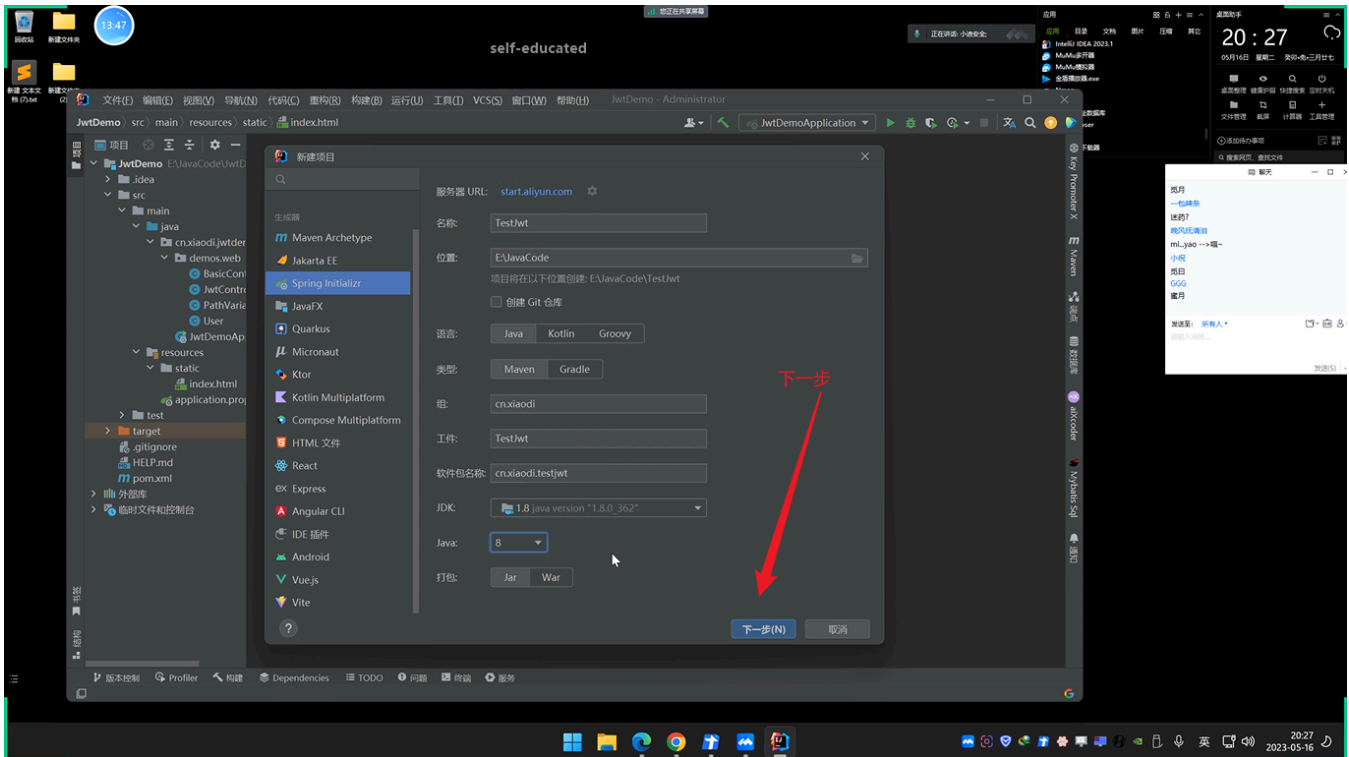
```
1 JWT(JSON web Token)是由服务端用加密算法对信息签名来保证其完整性和不可伪造;
2 Token里可以包含所有必要信息,这样服务端就无需保存任何关于用户或会话的信息;
3 JWT用于身份认证、会话维持等。由三部分组成,header、payload与signature。
4 参考: https://cloud.tencent.com/developer/article/2101634
5 1、引入依赖
6 <dependency>
7     <groupId>com.auth0</groupId>
8     <artifactId>java-jwt</artifactId>
9     <version>3.4.0</version>
10 </dependency>
11 2、创建JWT
12 JWT.create()
13 3、配置JWT
14 JWT.create()
15 //header
16 .withHeader(map)
17 //payload
18 .withClaim("userid",id)
19 .withClaim("username",user)
20 .withClaim("password",pass)
21 //signature
22 .sign(Algorithm.HMAC256("xiaodisec"));
23 4、解析JWT
24 //构建解密注册
```

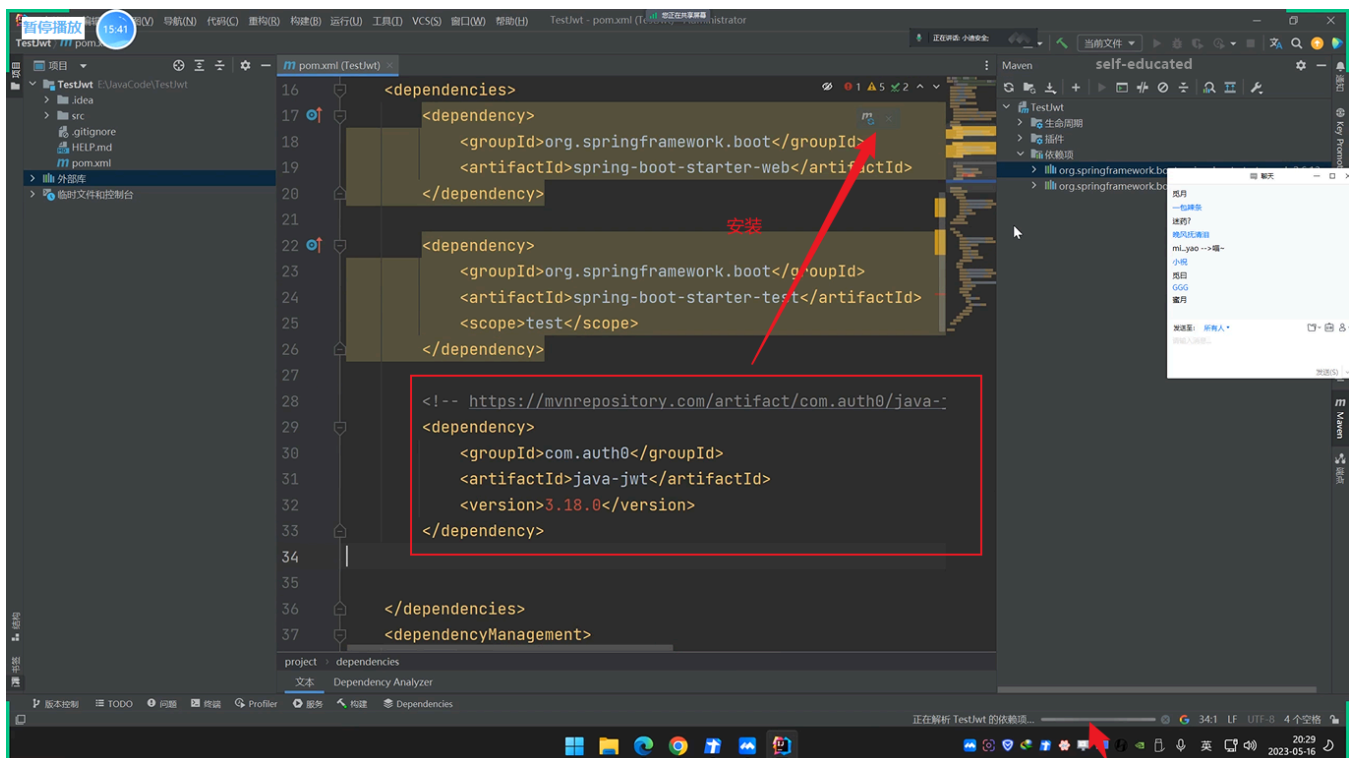
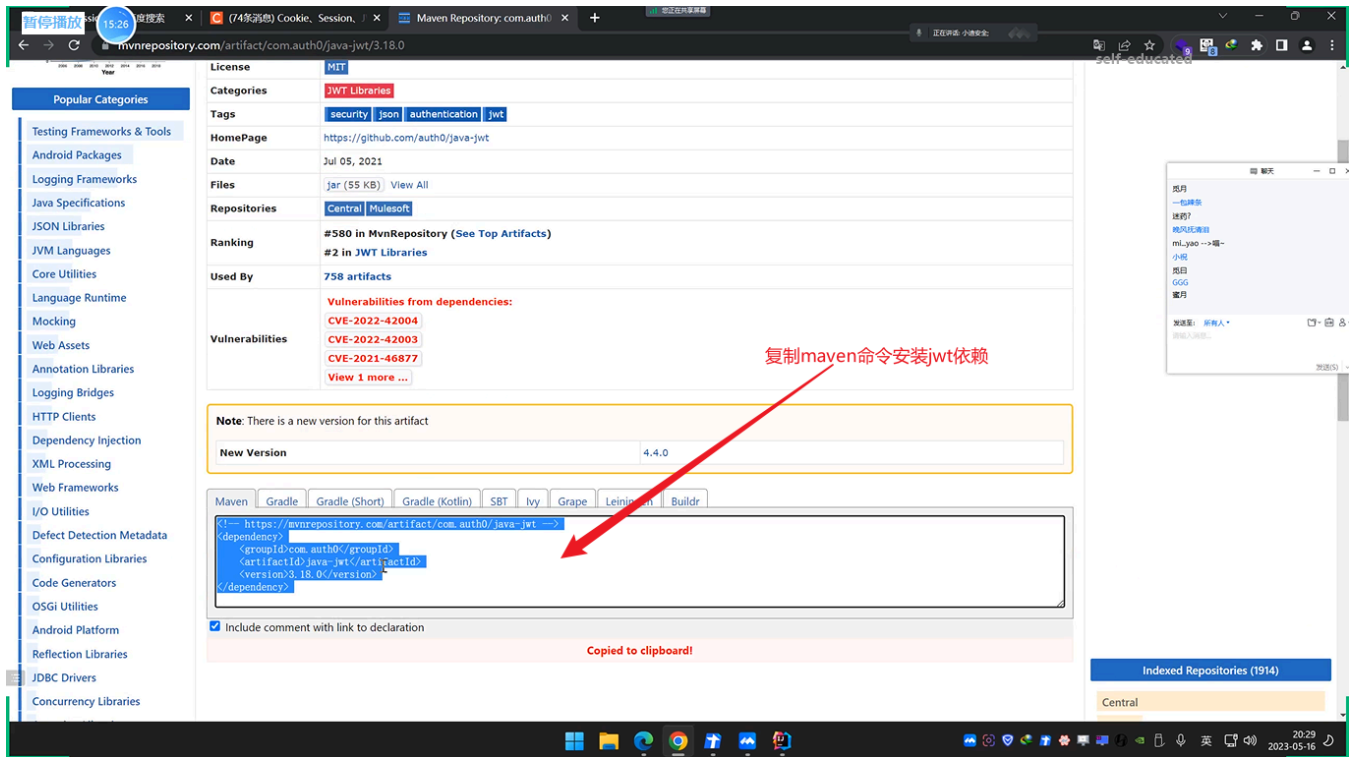
```

25  JWTVerifier jwt = JWT.require(Algorithm.HMAC256("xiaodisec")).build();
26  //解密注册数据
27  DecodedJWT verify = jwt.verify(jwtdata);
28  //提取解密数据
29  Integer userid = verify.getClaim("userid").asInt();
30  5、安全问题
31  参考: https://cloud.tencent.com/developer/article/2101634

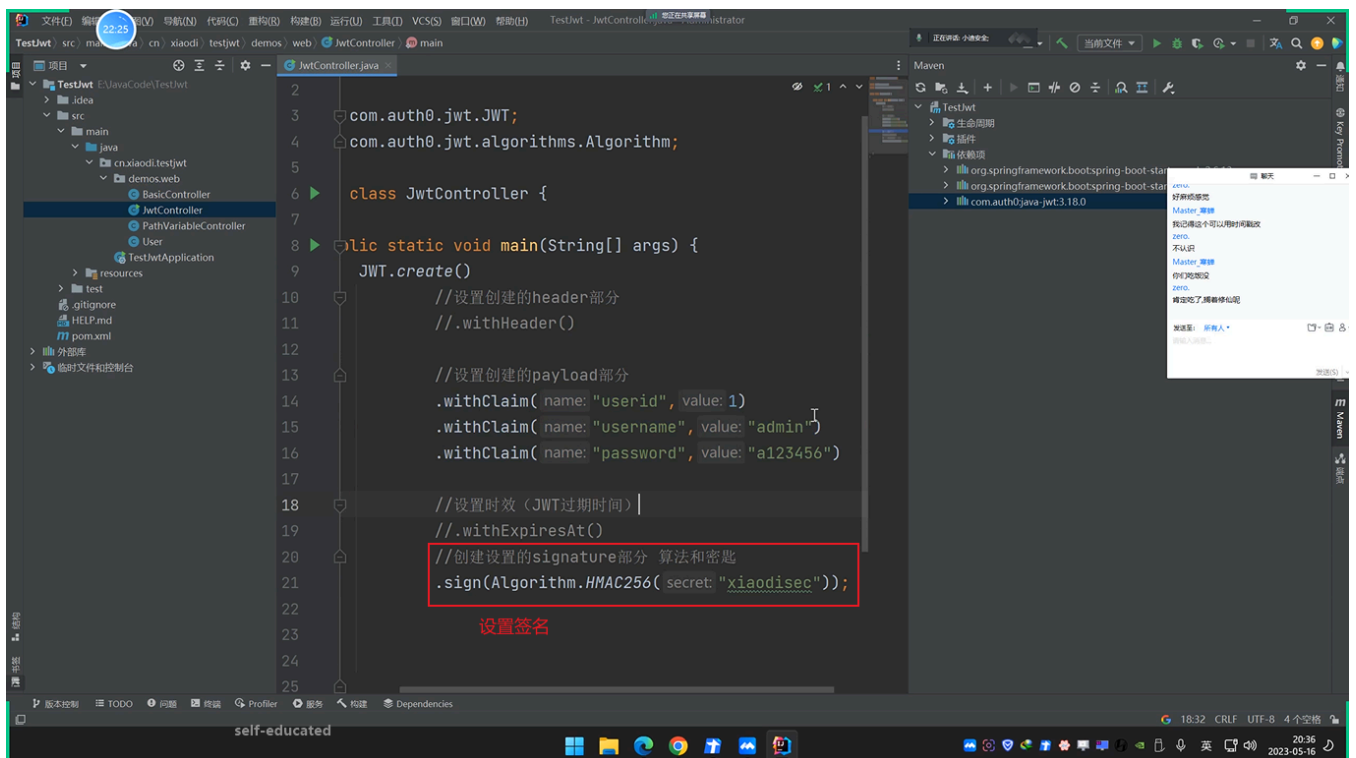
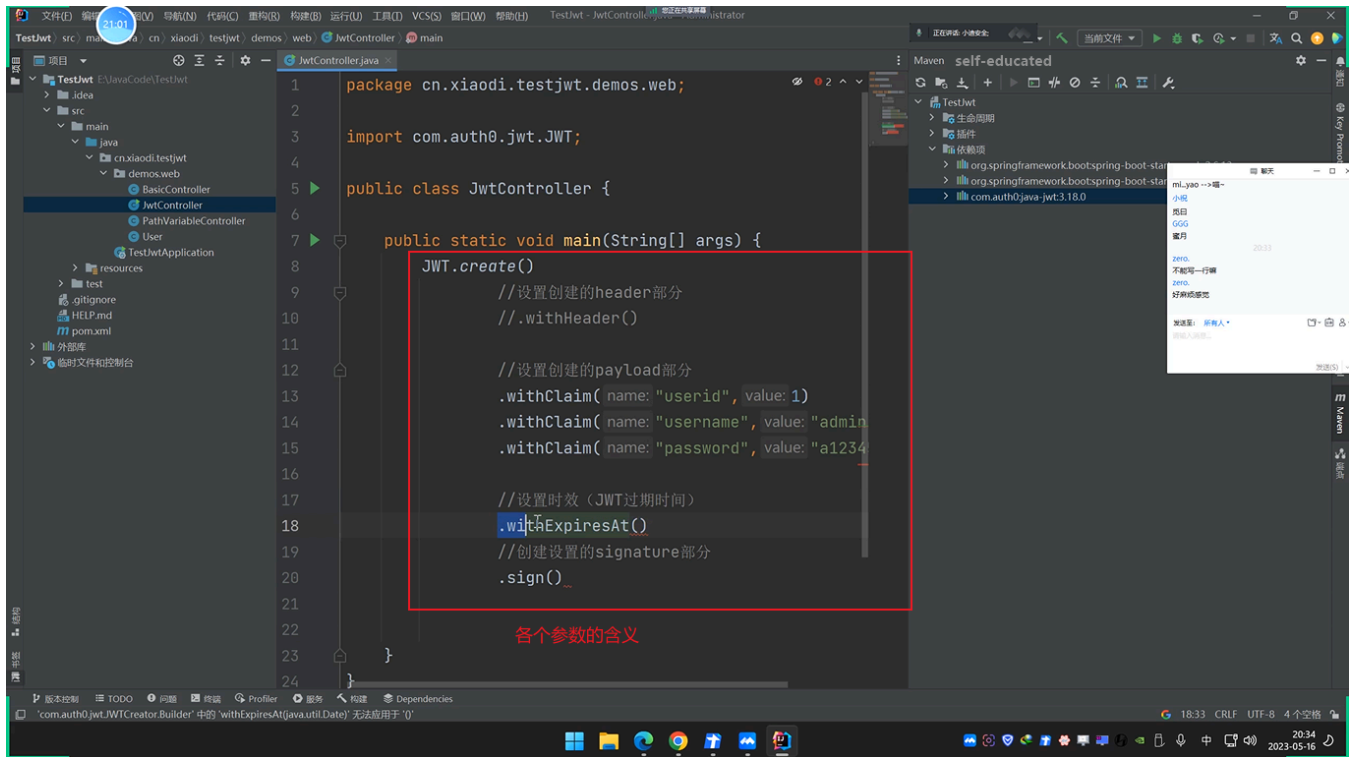
```

(1) 创建项目:





(2) 创建JWT&&配置JWT:



```
import com.auth0.jwt.algorithms.Algorithm;

public class JwtController {

    public static void main(String[] args) {
        String jwttoken = JWT.create()
            //设置创建的header部分
            // .withHeader()

            //设置创建的payload部分
            .withClaim( name: "userid", value: 1)
            .withClaim( name: "username", value: "admin")
            .withClaim( name: "password", value: "a123456")
            //设置时效 (JWT过期时间)
            // .withExpiresAt()

    }
}
```

打印的jwt数据

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJwYXNzd29yZCI6ImExMjM0NTYiLCJ1c2VyaWQiOiJEsInVzZXJ0eXN1IjoieWVWRtaW4ifQ.nkMIxHJKyGAHa3aDtTAy5_9j

(3) 解析JWT:

```
//模拟JWT身份的检测 jwt数据解密
0 个用法
public String jwtcheck(){
    String jwtdata="eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJwYXNzd29yZCI6ImExMjM0NTYiLCJ1c2VyaWQiOiJEsInVzZXJ0eXN1IjoieWVWRtaW4ifQ.nkMIxHJKyGAHa3aDtTAy5_9j";

    //构建解密注册
    JWTVerifier jwt = JWT.require(Algorithm.HMAC256( secret: "xiaodisec")).build();

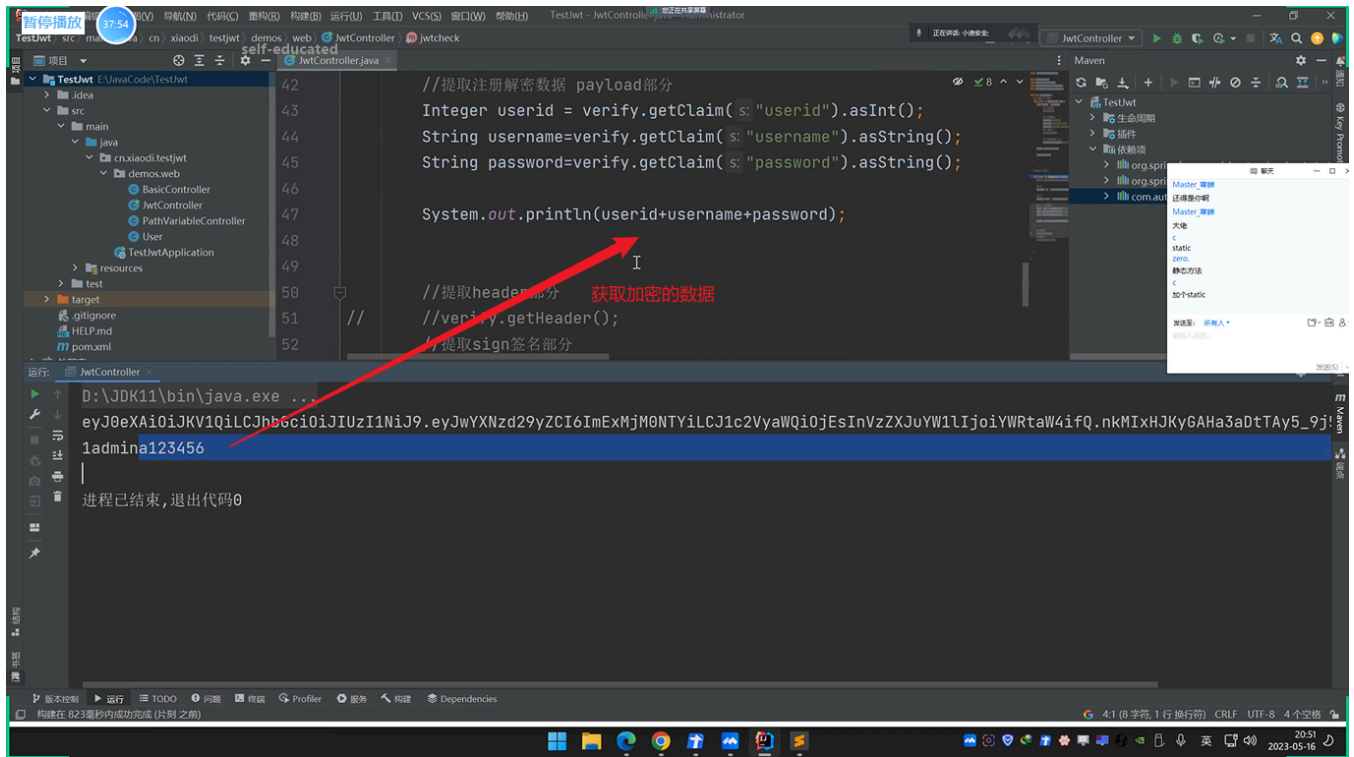
    //解密注册数据
    DecodedJWT verify = jwt.verify(jwtdata);

    //提取注册数据

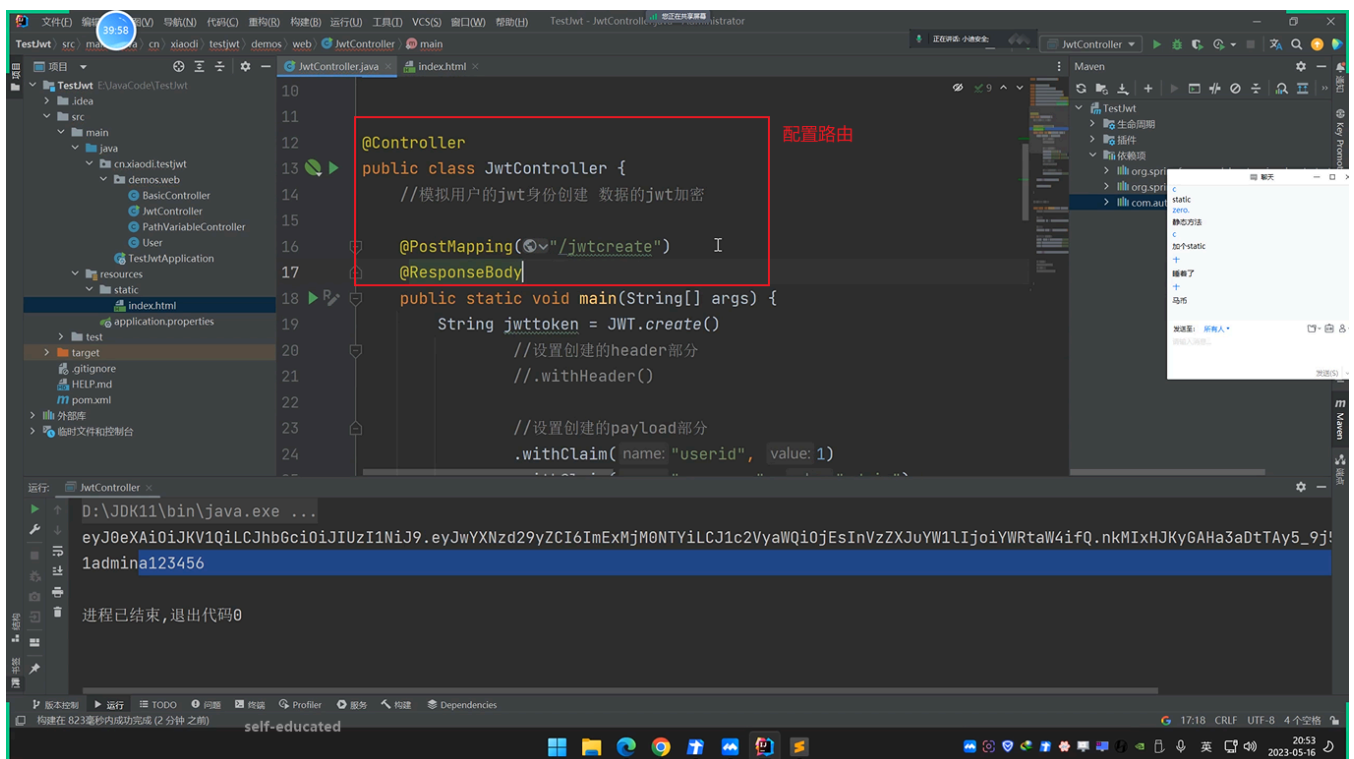
}
```

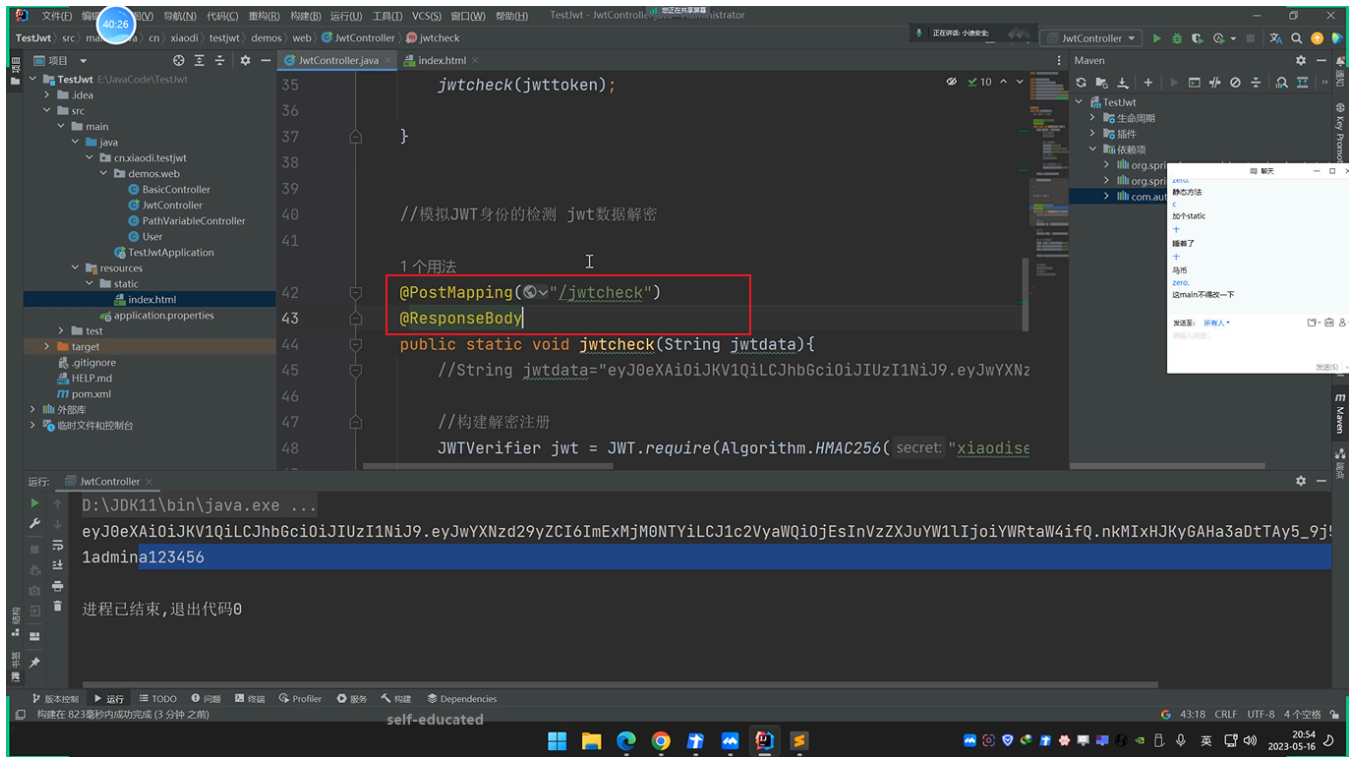
再创建一个函数用来校验jwt

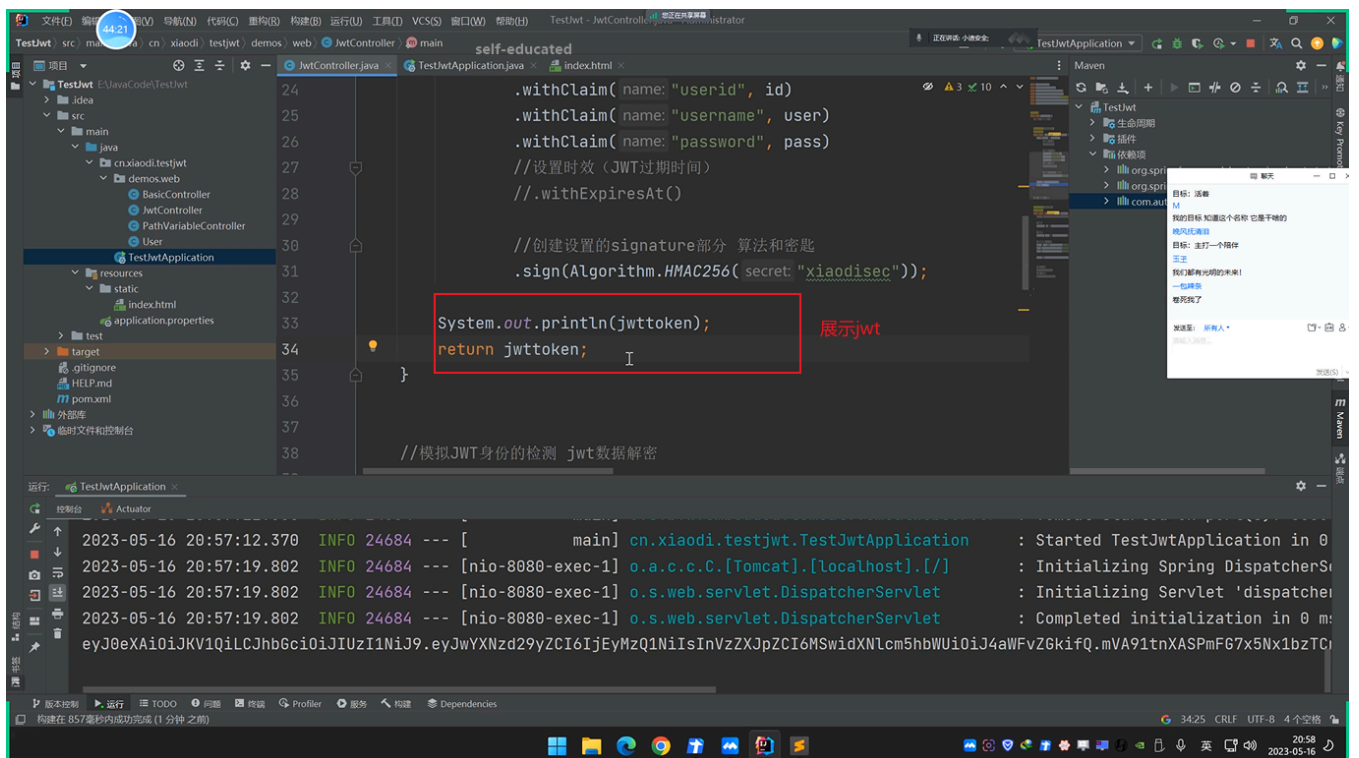
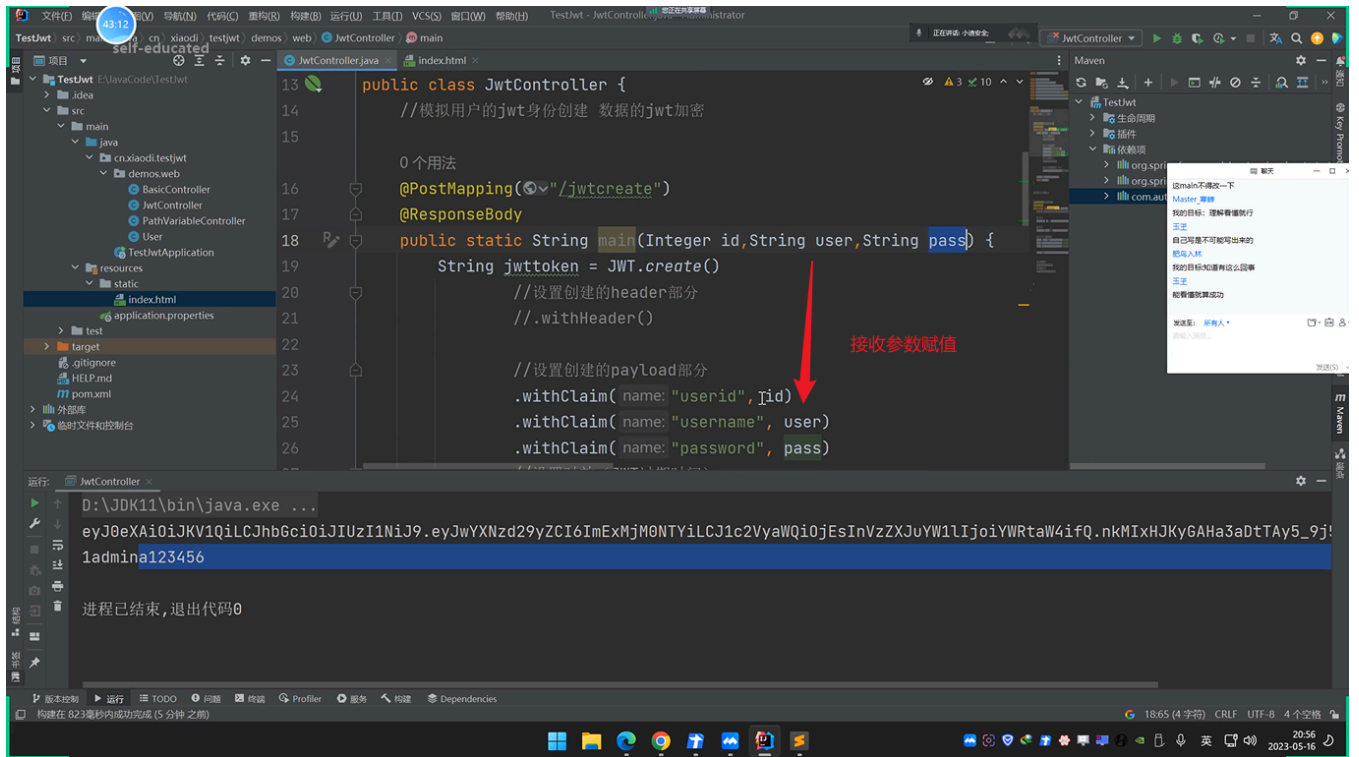
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJwYXNzd29yZCI6ImExMjM0NTYiLCJ1c2VyaWQiOiJEsInVzZXJ0eXN1IjoieWVWRtaW4ifQ.nkMIxHJKyGAHa3aDtTAy5_9j

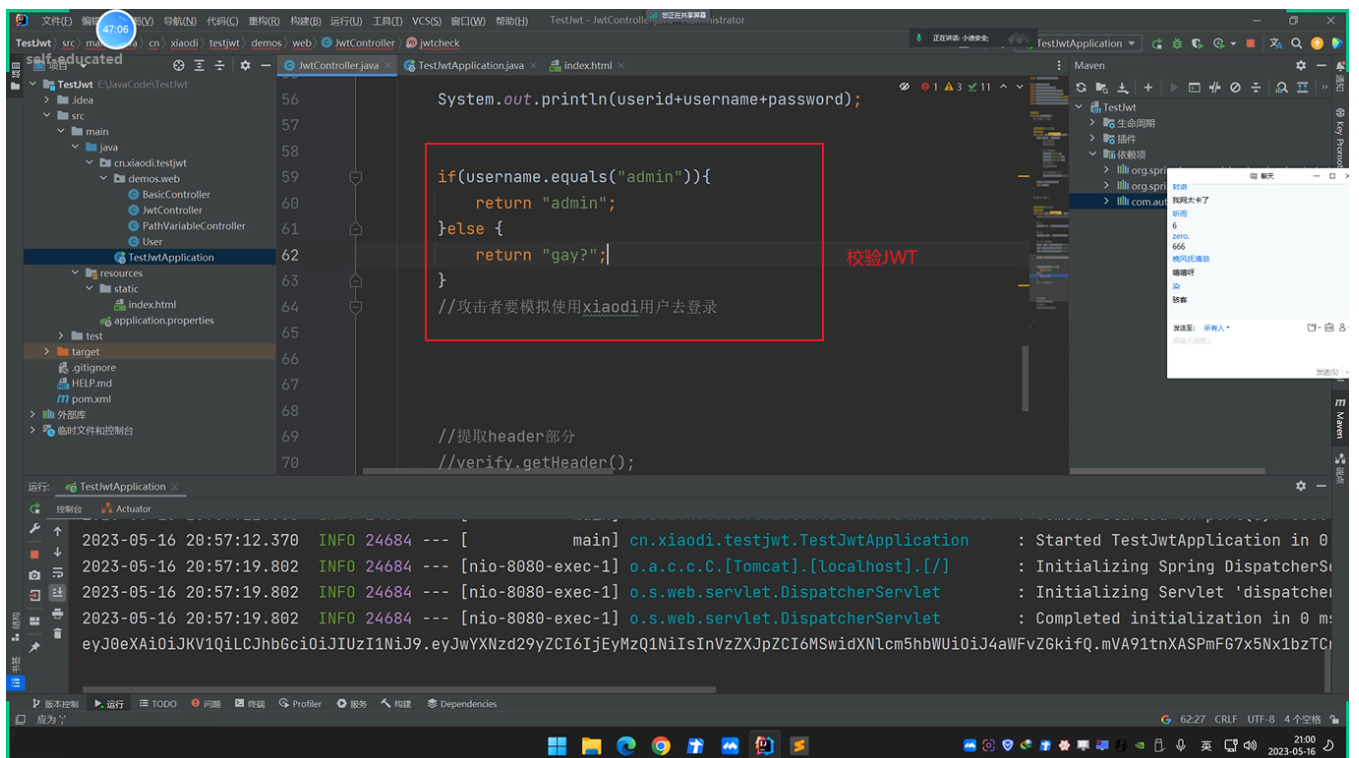
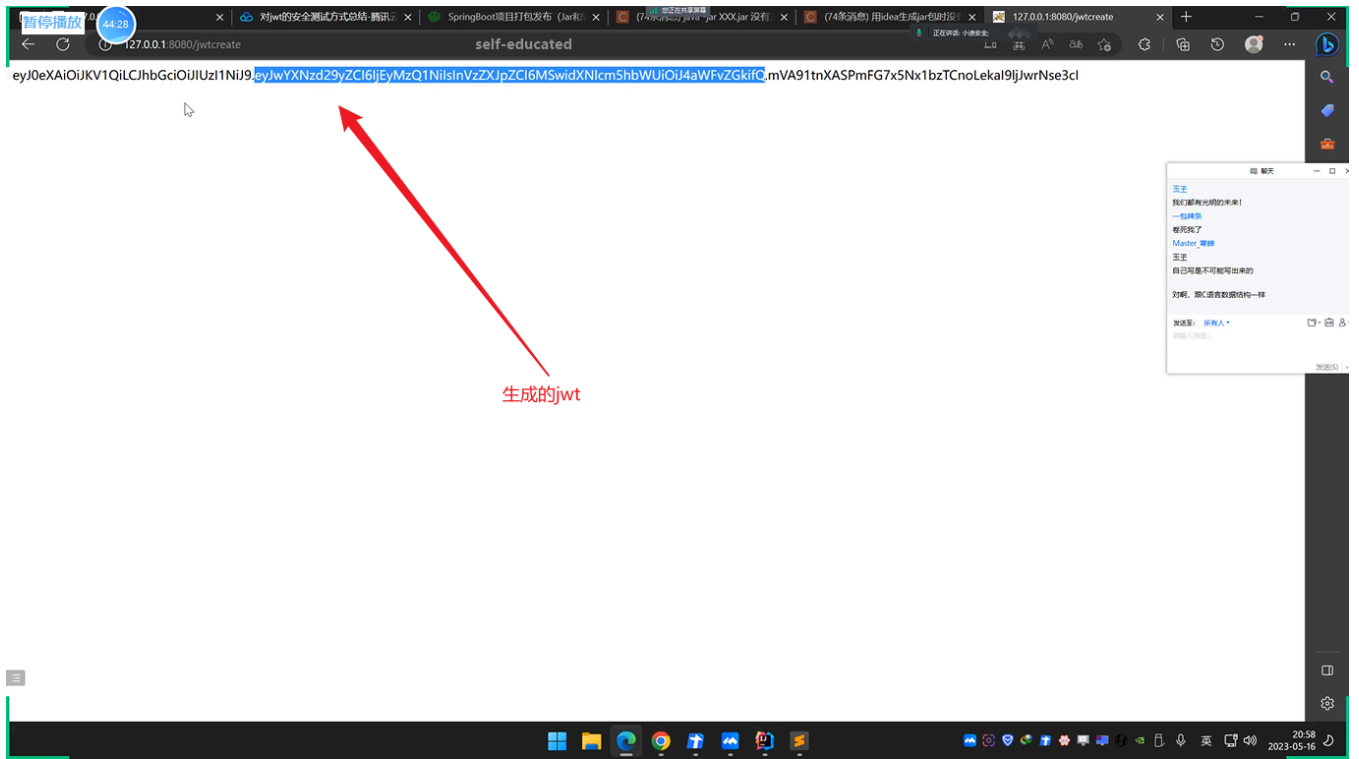


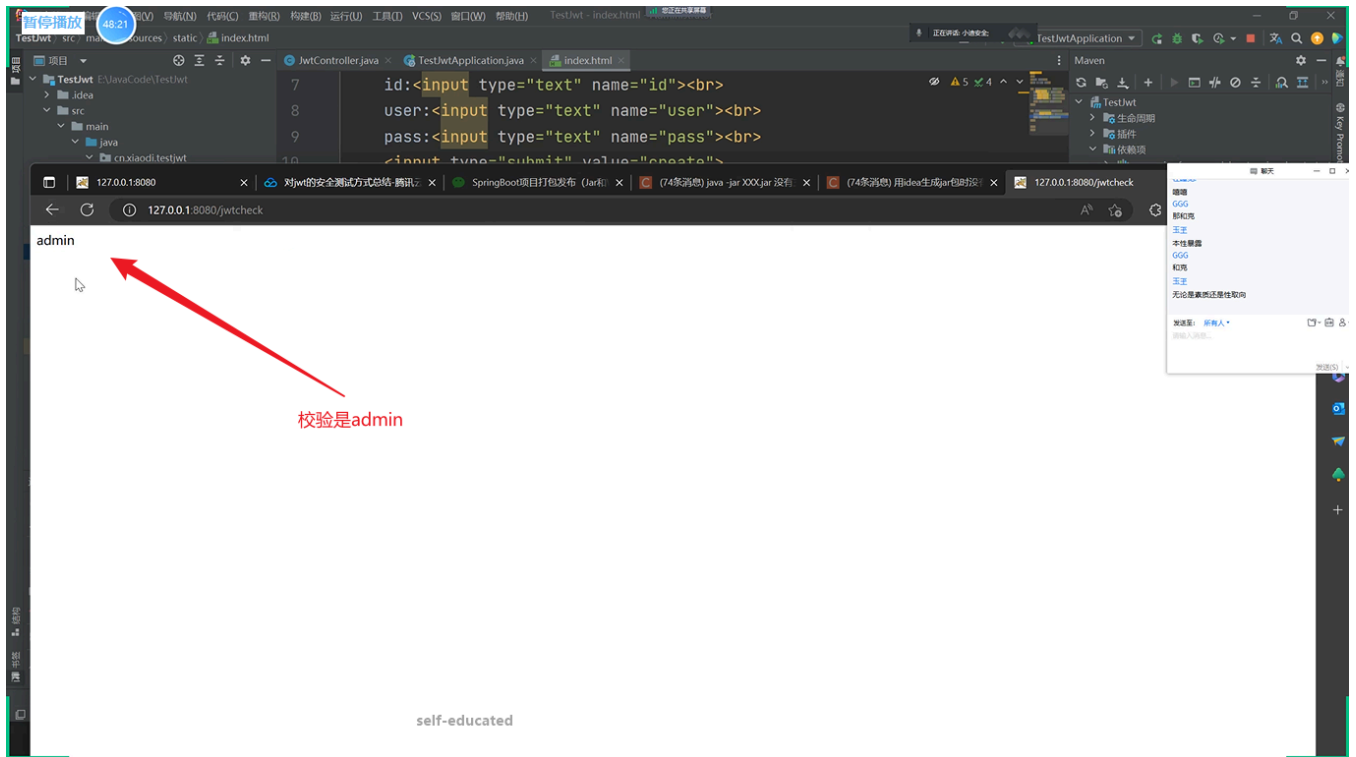
(4) 创建路由并进行JWT校验:



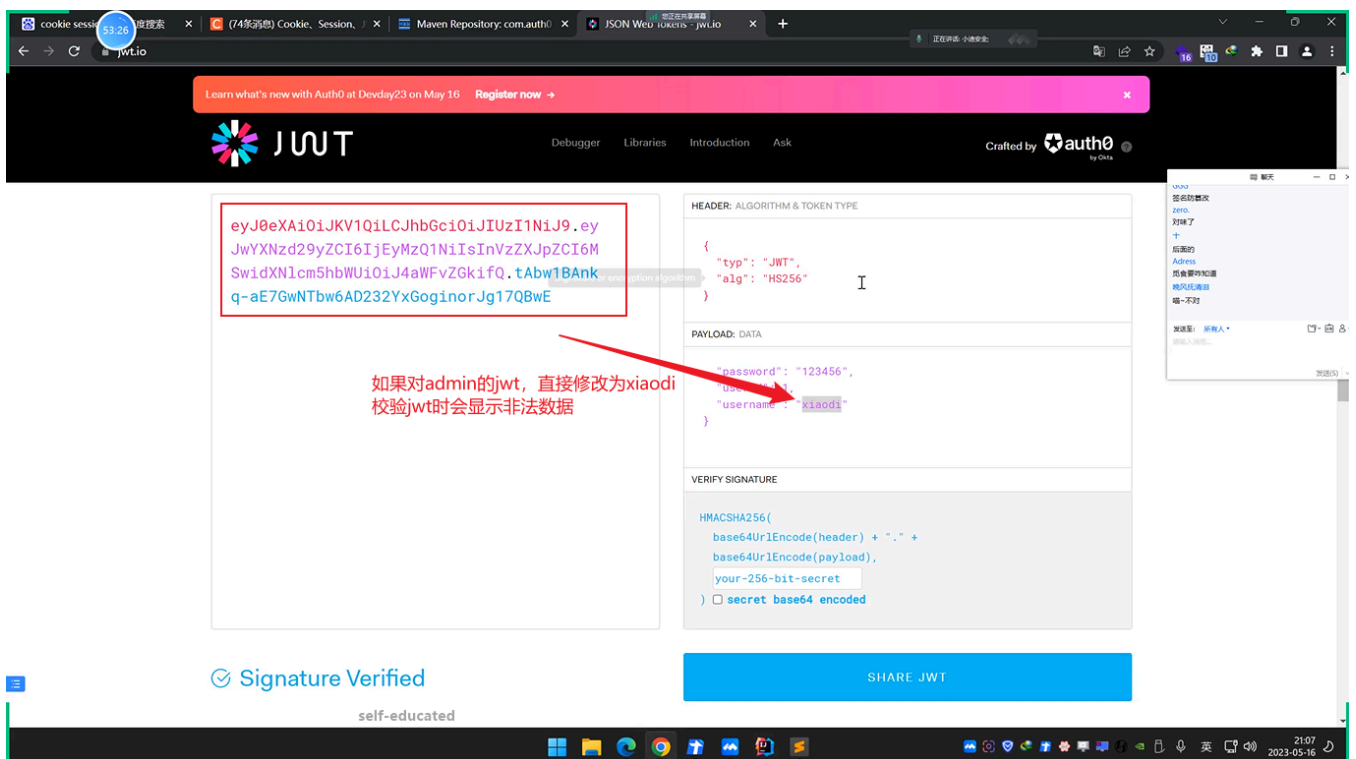






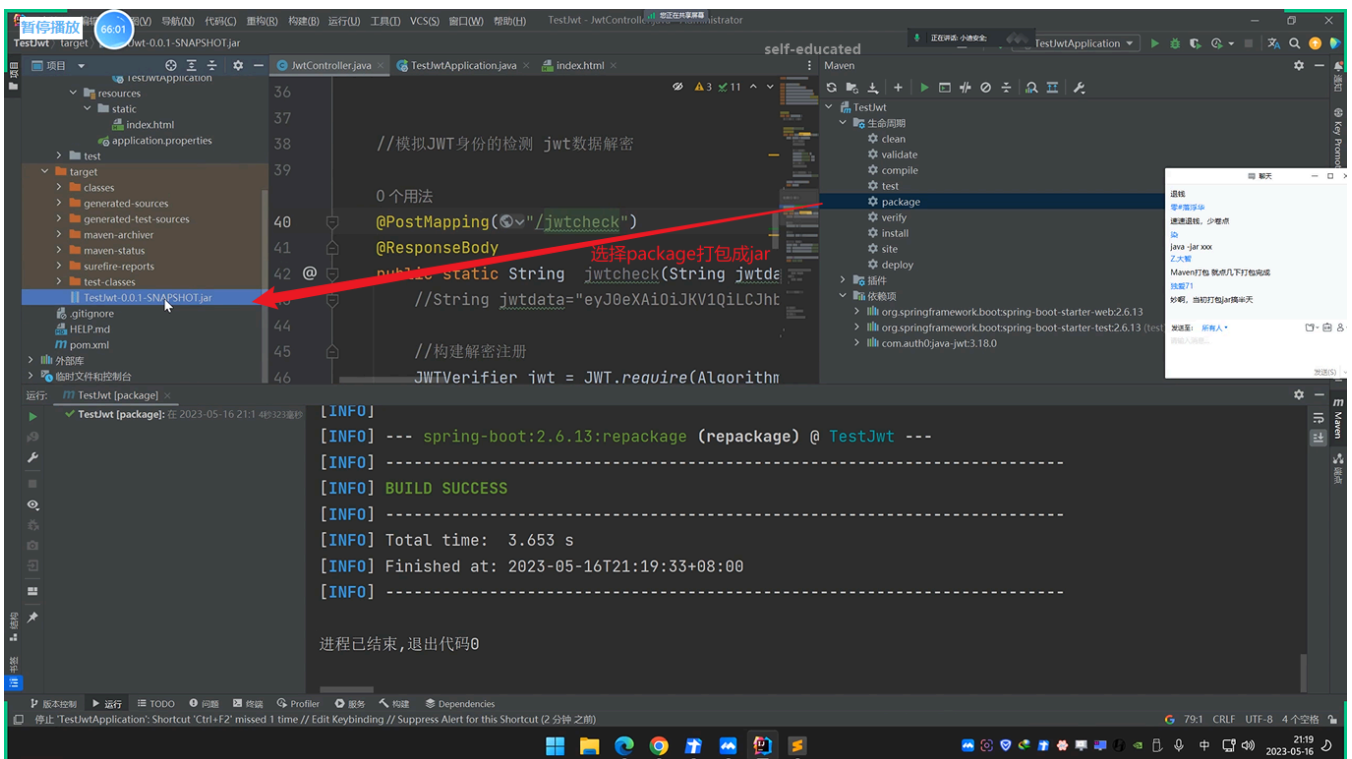
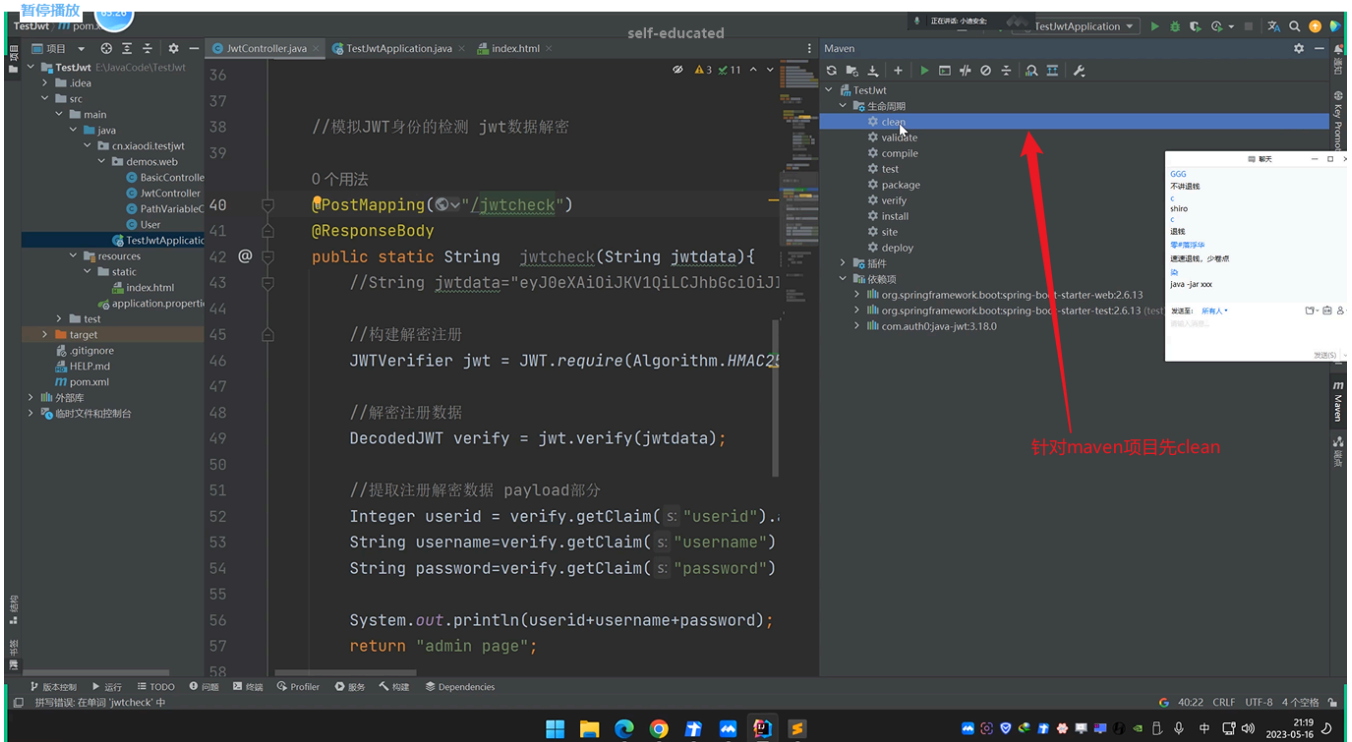


(5) 安全问题:

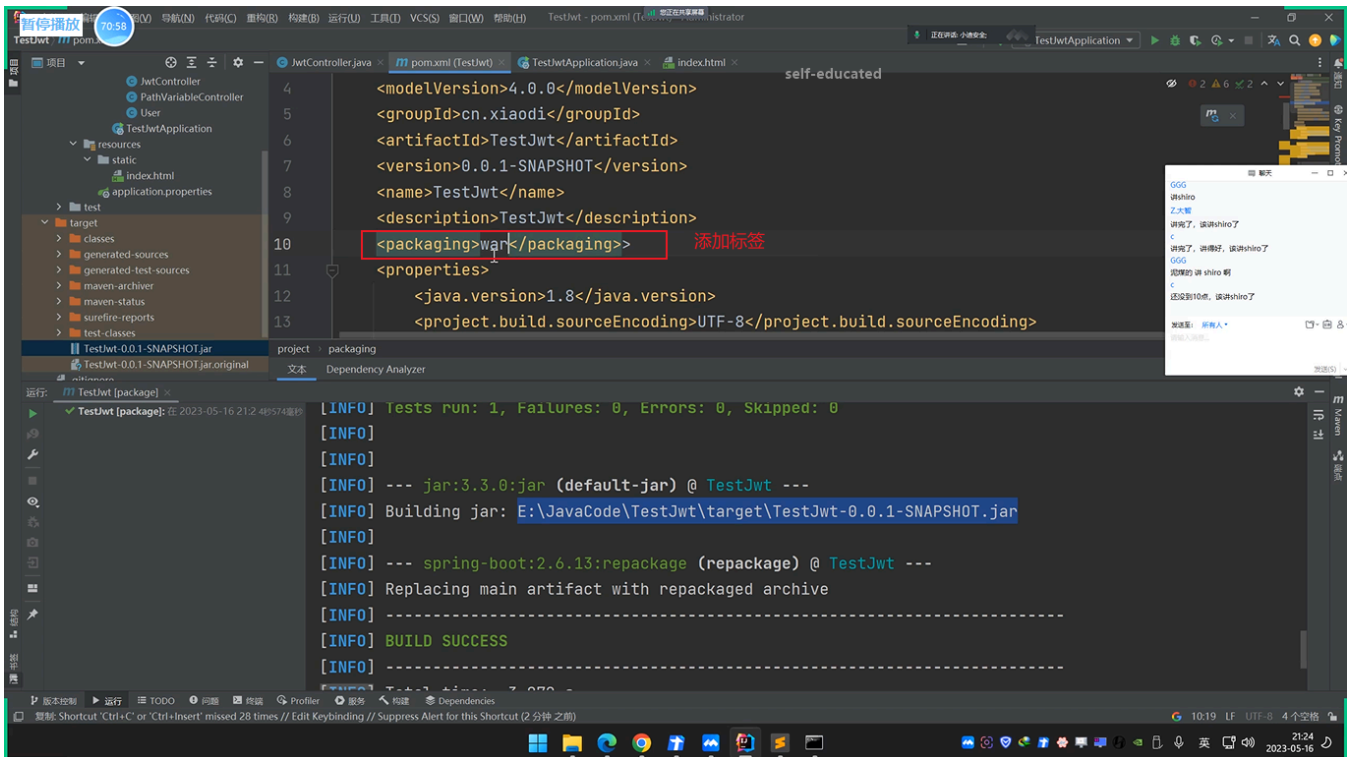



```
24     return builder.sources(TestSwaggerDemoApplication.class);
25 }
26 3、maven-clean-package
27 4、war放置tomcat后启动
28
29 JAVAEE源码架构:
30 无源码下载泄漏风险
31 源码泄漏也需反编译
```

(1) 使用jar打包项目:



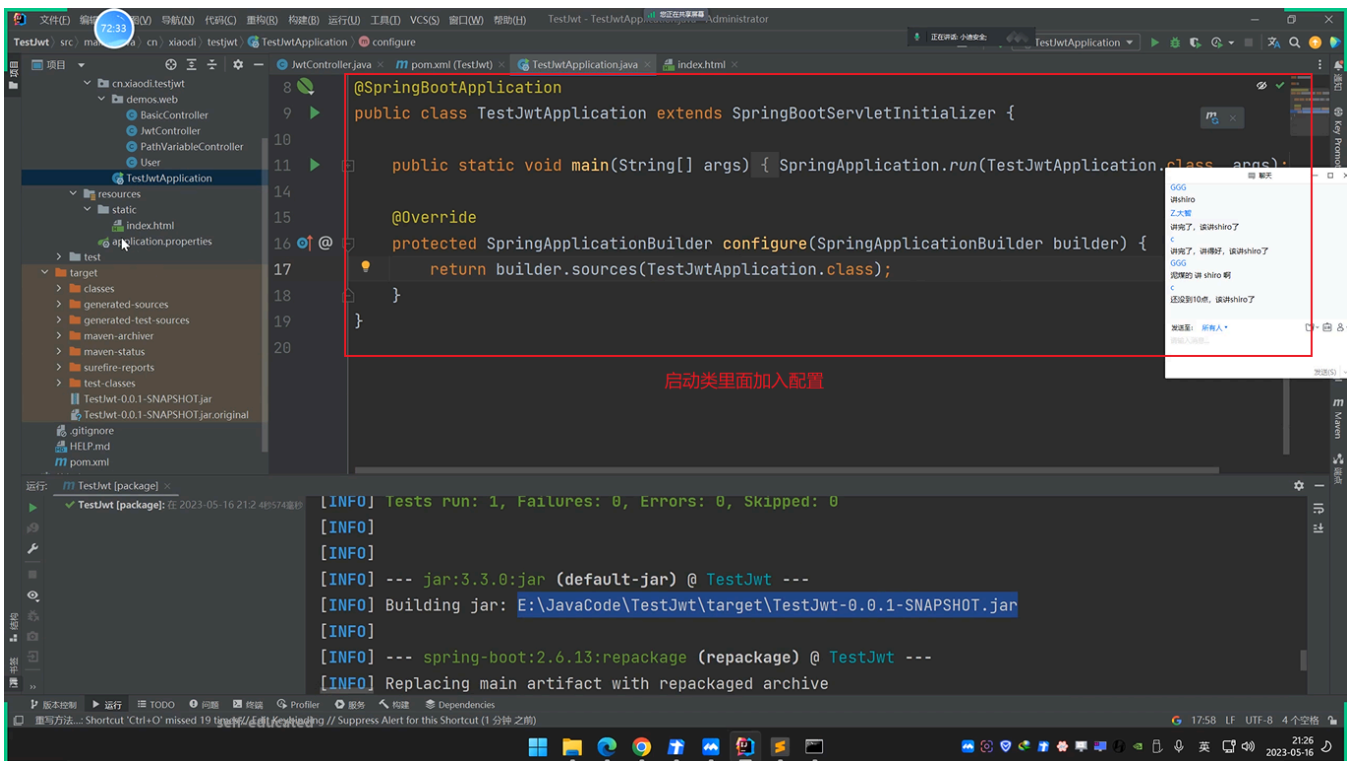
(2) 使用war打包项目:



The screenshot shows the IDE interface with the `pom.xml` file open. The `<packaging>war</packaging>` tag is highlighted with a red box and labeled "添加标签" (Add tag). The console output shows the build process for the `TestJwt` package, including the command `spring-boot:2.6.13:repackage` and the final message `BUILD SUCCESS`.

```
<modelVersion>4.0.0</modelVersion>
<groupId>cn.xiaodi</groupId>
<artifactId>TestJwt</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>TestJwt</name>
<description>TestJwt</description>
<packaging>war</packaging>
<properties>
  <java.version>1.8</java.version>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ TestJwt ---
[INFO] Building jar: E:\JavaCode\TestJwt\target\TestJwt-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:2.6.13:repackage (repackage) @ TestJwt ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[INFO] BUILD SUCCESS
[INFO]
```



The screenshot shows the IDE interface with the `TestJwtApplication.java` file open. The `@SpringBootApplication` annotation is highlighted with a red box and labeled "启动类里面加入配置" (Add configuration in the start class). The console output shows the build process for the `TestJwt` package, including the command `spring-boot:2.6.13:repackage` and the final message `BUILD SUCCESS`.

```
@SpringBootApplication
public class TestJwtApplication extends SpringServletInitializer {

    public static void main(String[] args) { SpringApplication.run(TestJwtApplication.class, args); }

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
        return builder.sources(TestJwtApplication.class);
    }
}
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ TestJwt ---
[INFO] Building jar: E:\JavaCode\TestJwt\target\TestJwt-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:2.6.13:repackage (repackage) @ TestJwt ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[INFO] BUILD SUCCESS
[INFO]
```